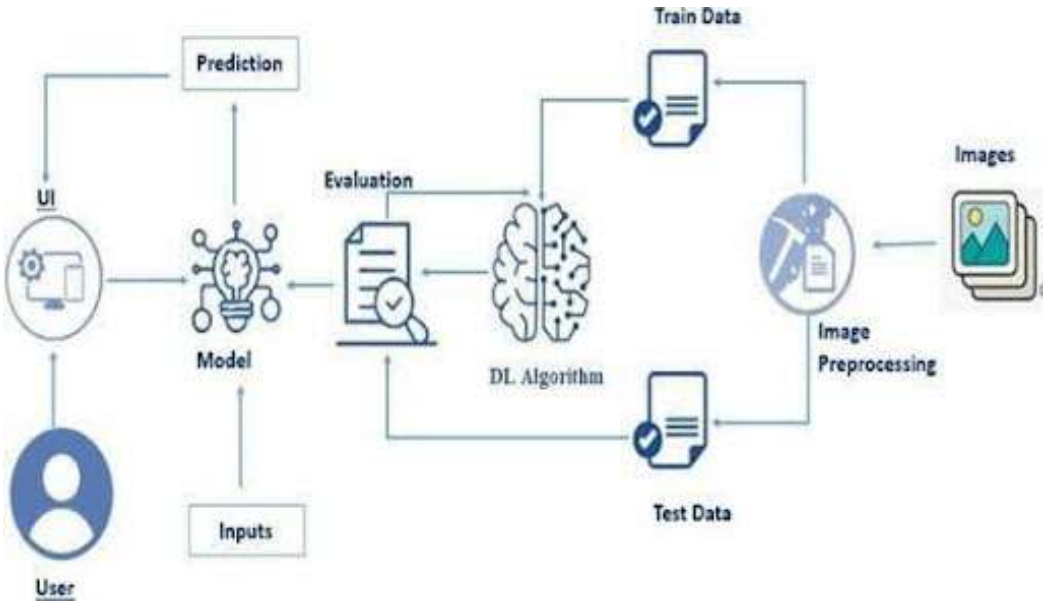
CleanTech: Transforming Waste Management with Transfer Learning

CleanTech aims to develop an intelligent and efficient model for classifying waste materials by employing transfer learning techniques. Utilizing a dataset of thousands of annotated waste images, categorized into distinct classes such as Recyclable,Biodegradable and Trash the project leverages pre-trained convolutional neural networks (CNNs) to expedite training and improve classification accuracy. Transfer learning enables the model to benefit from pre-existing knowledge of image features, significantly enhancing its performance while reducing computational costs. This approach provides a scalable and reliable tool for municipalities, recycling units, and waste management companies, ensuring precise and efficient waste segregation.

Technical Architecture:



Prerequisites

To complete this project, you must require the following software, concepts, and packages

- Anaconda Navigator:
    - Refer to the link below to download Anaconda Navigator
- Python packages:
    - Open anaconda prompt as administrator
    - Type “pip install numpy” and click enter.
    - Type “pip install pandas” and click enter.
    - Type “pip install scikit-learn” and click enter.
    - Type ”pip install matplotlib” and click enter.
    - Type ”pip install scipy” and click enter.
    - Type ”pip install seaborn” and click enter.
    - Type ”pip install tensorflow” and click enter.
    - Type “pip install Flask” and click enter.

Prior Knowledge

You must have prior knowledge of the following topics to complete this project.

DL Concepts

- Neural Networks:: https://www.analyticsvidhya.com/blog/2020/02/cnn-vs-rnn-vs-mlp-analyzing-3-types-of-neural-networks-in-deep-learning/
- Deep Learning Frameworks:: https://www.knowledgehut.com/blog/data-science/pytorch-vs-tensorflow
- Transfer Learning: https://towardsdatascience.com/a-demonstration-of-transfer-learning-of-vgg-convolutional-neural-network-pre-trained-model-with-c9f5b8b1ab0a
- VGG16: https://www.geeksforgeeks.org/vgg-16-cnn-model/
- Convolutional Neural Networks (CNNs): https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/ s://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning
- Overfitting and Regularization: https://www.analyticsvidhya.com/blog/2021/07/prevent-overfitting-using-regularization-techniques/
- Optimizers: https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-deep-learning-optimizers/
- Flask Basics: https://www.youtube.com/watch?v=lj4I_CvBnt0

Project Objectives

By the end of this project, you will:

- Know fundamental concepts and techniques used for Deep Learning.
- Gain a broad understanding of data.
- Have knowledge of pre-processing the data/transformation techniques on outliers and some visualization concepts.

Project Flow:

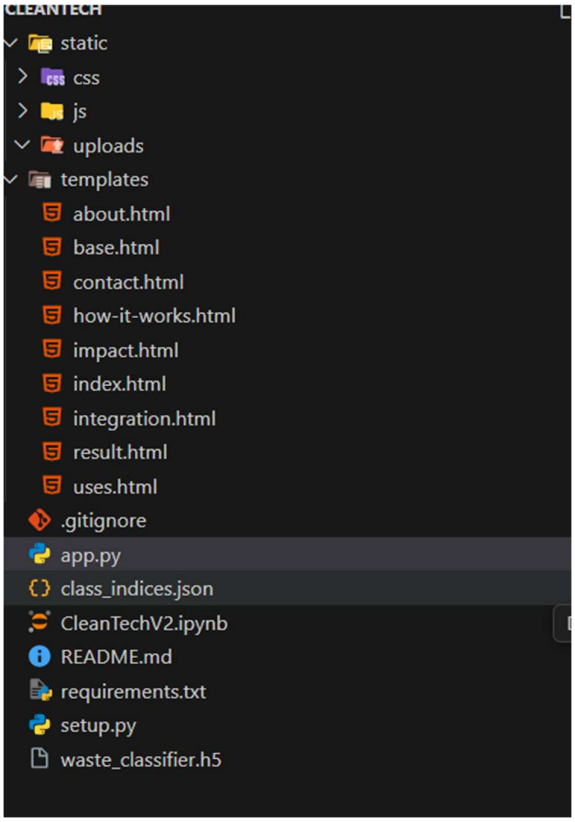
- User interacts with the UI to enter the input.
- Entered input is analysed by the model which is integrated.
- Once model analyses the input the prediction is showcased on the UI

To accomplish this, we have to complete all the activities listed below,

- Define Problem / Problem Understanding
    - Specify the business problem
    - Business requirements
    - Literature Survey
    - Social or Business Impact.
- Data Collection & Preparation
    - Collect the dataset
    - Data Preparation
- Splitting &Model Building
    - Training the model in multiple algorithms
    - Testing the model
- Performance Testing
    - Testing model with multiple evaluation metrics
    - Comparing model accuracy before & after applying hyperparameter tuning
- Model Deployment
    - Save the best model
    - Integrate with Web Framework
- Project Demonstration & Documentation
    - Record explanation Video for project end to end solution
    - Project Documentation-Step by step project development procedure

Project Structure

Create the Project folder which contains files as shown below



- We are building a flask application which needs HTML pages stored in the templates folder and a python script app.py for scripting.
- waste_classifier.h5 is our saved model. Further we will use this model for flask integration.
- Data Folder contains the Dataset used
- The Notebook file contains procedure for building the model.

1. Define Problem / Problem Understanding:

This initial phase establishes the foundation of the CleanTech project by defining the core business problem, identifying key requirements, conducting a literature survey to select the appropriate technology, and outlining the potential social and business impact of the solution.

1.1 :Business Problem:

The primary challenge addressed by CleanTech is the **inefficiency, high cost, and inaccuracy of traditional waste segregation methods**. Manual sorting is labor-intensive and slow, leading to suboptimal recycling rates and an increasing burden on landfills. An automated system is needed to streamline this process effectively.

1.2 :Business Requirements:

To solve this problem, the proposed system must meet the following requirements:

- **Accuracy:** Achieve high precision in classifying different types of waste materials to ensure effective segregation.
- **Efficiency:** Perform classification in real-time to be viable for automated sorting systems and smart bins.
- **Scalability:** Be adaptable for various stakeholders, including municipalities, private waste management companies, and recycling facilities.
- **Cost-Effectiveness:** Reduce the need for manual labor and leverage computationally efficient technology to lower operational costs.

1.2 :Literature Survey

The technical approach selected for this project is **transfer learning** using pre-trained **Convolutional Neural Networks (CNNs)**. This method is ideal for image classification tasks as it leverages the powerful feature-extraction capabilities of models already trained on massive datasets. By using transfer learning, CleanTech can achieve high accuracy with significantly less training data and computational resources compared to training a model from scratch.

1.4 :Social and Business Impact

The successful implementation of CleanTech is projected to have a wide-ranging impact across multiple sectors:

- **Smart Waste Management:** Revolutionize recycling facilities by enabling automated, high-speed sorting systems that reduce manual effort and improve the purity of recycled materials.

- **Smart Cities:** Integrate with IoT-enabled smart bins to provide real-time data on waste levels and types, allowing municipalities to optimize collection routes and schedules.
- **Environmental Education:** Serve as an interactive educational tool for communities and schools, promoting environmental awareness and teaching correct waste segregation practices to foster sustainable habits.

2.Data Collection and Preparation:

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

2.1 Data Collection:

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.
In this project, we have used 3 classes of biodegradable, recyclable and trash images data. This data is downloaded from kaggle.com or can be connected by using API. Please refer to the link given below to download the dataset.
Link:Dataset
As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analyzing techniques.
Note: There are several techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

2.2 Data Visualization

The provided Python code imports necessary libraries and modules for image manipulation. It selects a random image file from a specified folder path. Then, it displays the randomly selected image using IPython's Image module. This code is useful for showcasing random images from a directory for various purposes like data exploration or testing image processing algorithms.

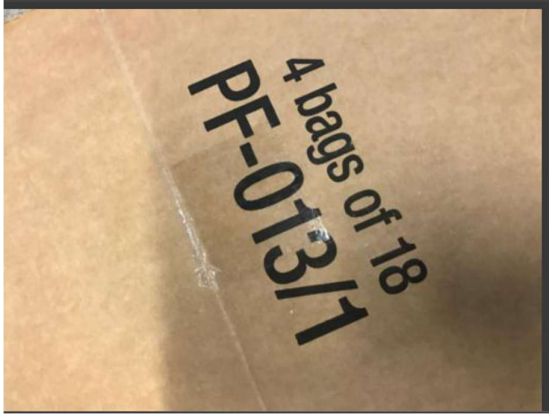
```python
import random
from IPython.display import Image, display
#a specify the path to your image Folder folder_path/content/output_dataset/train/Biodegradable Images
# Replace with the actual path to your image folder
folder_path = '/content/output_dataset/train/Biodegradable Images'

#List all files in the folder
image_files = [f for f in os.listdir(folder_path) if f.endswith((".jpg", ".png", ".jpeg"))]
# Select a random image from the list
selected_image = random.choice(image_files)
#Display the randomly selected image
image_path = os.path.join(folder_path, selected_image)
display(Image(filename=image_path))
```

In the above code, I used class biodegradable class for prediction, This code randomly selects an image file from a specified folder (folder_path) containing JPEG, PNG, or JPEG files, and then displays the selected image using IPython's display function. It utilizes Python's OS and random modules for file manipulation and random selection, respectively. And It has predicted correctly as biodegradable image.

```python
folder_path = '/content/output_dataset/train/Recyclable Images' # Example path, replace with your desired folder

#List all files in the folder
image_files = [f for f in os.listdir(folder_path) if f.endswith((".jpg", ".png", ".jpeg"))]
# Select a random image from the list
selected_image = random.choice(image_files)
#Display the randomly selected image
image_path = os.path.join(folder_path, selected_image)
display(Image(filename=image_path))
```



In the above code, I used recyclable class for prediction, This code randomly selects an image file from a specified folder (folder_path) containing JPEG, PNG, or JPEG files, and then displays the selected image using IPython's display function. It utilizes Python's OS and random modules for file manipulation and random selection, respectively. And It has predicted correctly as recyclable image.

```
folder_path = '/content/output_dataset/train/Trash Images' # Example path,
#List all files in the folder
image_files = [f for f in os.listdir(folder_path) if f.endswith((".jpg", ".png", ".jpeg"))]
# Select a random image from the list
selected_image = random.choice(image_files)
#Display the randomly selected image
image_path = os.path.join(folder_path, selected_image)
display(Image(filename=image_path))
```

In the above code, I used trash class for prediction, This code randomly selects an image file from a specified folder (folder_path) containing JPEG, PNG, or JPEG files, and then displays the selected image using IPython's display function. It utilizes Python's OS and random modules for file manipulation and random selection, respectively. And It has predicted correctly as trash image.

2.3:Data Augmentation:

Data augmentation is a technique commonly employed in machine learning, particularly in computer vision tasks such as image classification, including projects like the healthy vs rotten Classification in fruits and vegetables. The primary objective of data augmentation is to artificially expand the size of the training dataset by applying various transformations to the existing images, thereby increasing the diversity and robustness of the data available for model training. This approach is particularly beneficial when working with limited labeled data.

```
dataset_dir = '/content/output_dataset'
train_dir = os.path.join(dataset_dir, 'train')
val_dir = os.path.join(dataset_dir, 'val')
test_dir = os.path.join(dataset_dir, 'test')

#Define image size expected by the pre-trained model
IMG_SIZE = (224, 224) # Common size for many models like Reshet, VGG, MobileNet

#Create ImageDataGenerators for resizing and augmenting the images
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
val_test_datagen = ImageDataGenerator (rescale=1./255)

#Load and resize the images from directories
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=32,
    class_mode='categorical' # Assuming categorical classification for multiple classes
)
```

```
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=32,
    class_mode='categorical' # Assuming categorical classification for multiple classes
)
val_generator = val_test_datagen.flow_from_directory(
    val_dir,
    target_size=IMG_SIZE,
    batch_size=32,
    class_mode='categorical'
)
test_generator = val_test_datagen.flow_from_directory(
    test_dir,
    target_size=IMG_SIZE,
    batch_size=32,
    class_mode='categorical',
    shuffle=False # Do not shuffle test data
)
#Print class indices for reference
print(train_generator.class_indices)
print(val_generator.class_indices)
print(test_generator.class_indices)
```

```
Found 234 images belonging to 3 classes.
Found 78 images belonging to 3 classes.
Found 78 images belonging to 3 classes.
```

In this section, the dataset is divided into three parts — **training**, **validation**, and **testing** — to properly train and evaluate the model. The images are loaded from their respective directories using the ImageDataGenerator class, which is responsible for preprocessing and augmenting the data.

The **training data generator** applies several augmentation techniques such as rotation, width and height shifting, shearing, zooming, and horizontal flipping. These transformations artificially expand the training dataset, making the model more robust and less likely to overfit. Additionally, all pixel values are rescaled to the range [0, 1] for normalization.

For the **validation** and **test** datasets, only rescaling is performed to maintain consistency while evaluating model performance on unaltered images. Each image is resized to **224 × 224 pixels**, which is the standard input size for many pre-trained CNN architectures like **ResNet**, **VGG**, and **MobileNet**.

Finally, the class indices are printed to display the mapping between class names and their corresponding numeric labels. This step helps verify that the data has been correctly categorized before model training begins.

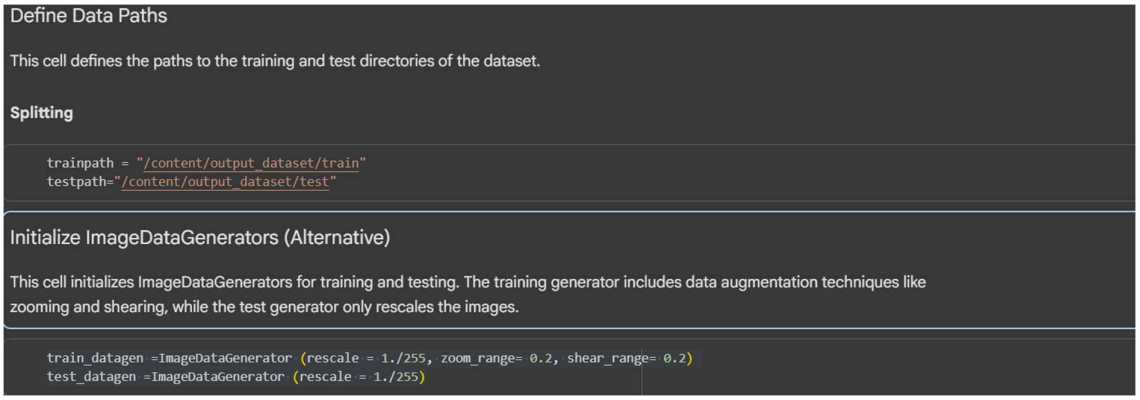
3.Splitting Data and Model Building

3.1:Train-Test-Split:

In this project, the dataset is divided into training and testing subsets to evaluate model performance effectively.

- The training data is stored in the path: "/content/output_dataset/train" and is used to train the model so it can learn patterns and features.
- The testing data is stored in the path: "/content/output_dataset/test" and is used to test the model's accuracy and generalization on unseen data.

This split ensures that the model doesn't just memorize the data but learns to make predictions effectively on new inputs.



3.2:Model Building:

Vgg16 Transfer-Learning Model:

The VGG16-based neural network is created using a pre-trained VGG16 architecture with frozen weights. The model is built sequentially, incorporating the VGG16 base, a flattening layer, dropout for regularization, and a dense layer with SoftMax activation for classification into three categories. The model is compiled using the Adam optimizer and sparse categorical cross-entropy loss. During training, which spans 10 epochs, a generator is employed for the training data, and validation is conducted, incorporating call-backs such as Model Checkpoint and Early Stopping. The best-performing model is saved as " waste_classifier.h5 " for potential future use. The model summary provides an overview of the architecture, showcasing the layers and parameters involved.

```
#model
vgg = VGG16(include_top=False, input_shape=(224,224,3))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Print VGG16 Layers

This cell iterates through and prints the names of the layers in the loaded VGG16 model.

```
for layers in vgg.layers:
  print(layers)
```

```
<InputLayer name=input_layer, built=True>
<Conv2D name=block1_conv1, built=True>
<Conv2D name=block1_conv2, built=True>
<MaxPooling2D name=block1_pool, built=True>
<Conv2D name=block2_conv1, built=True>
<Conv2D name=block2_conv2, built=True>
<MaxPooling2D name=block2_pool, built=True>
<Conv2D name=block3_conv1, built=True>
<Conv2D name=block3_conv2, built=True>
<Conv2D name=block3_conv3, built=True>
<MaxPooling2D name=block3_pool, built=True>
<Conv2D name=block4_conv1, built=True>
<Conv2D name=block4_conv2, built=True>
<Conv2D name=block4_conv3, built=True>
<MaxPooling2D name=block4_pool, built=True>
```

```
<Conv2D name=block4_conv1, built=True>
<Conv2D name=block4_conv2, built=True>
<Conv2D name=block4_conv3, built=True>
<MaxPooling2D name=block4_pool, built=True>
<Conv2D name=block5_conv1, built=True>
<Conv2D name=block5_conv2, built=True>
<Conv2D name=block5_conv3, built=True>
<MaxPooling2D name=block5_pool, built=True>
```

```
start coding or generate with AI.
```

Add Custom Classification Layer

This cell adds a Flatten layer followed by a Dense layer with 3 units (for the 3 classes: Biodegradable, Recyclable, Trash) and a 'softmax' activation function to the output of the VGG16 model. This creates the new classification head for our model.

```
output=Dense(3,activation='softmax')(Flatten()(vgg.output)) #
```

Create the Final Model

This cell creates the final model by combining the pre-trained VGG16 base model with the newly added classification layers.

```
vgg16=Model(vgg.input,output)
```

A **Flatten** layer is then added to convert the extracted feature maps into a one-dimensional vector, followed by a **Dense** layer with **3 output neurons** and a **softmax activation function**. This final layer is responsible for classifying images into three categories based on the dataset used.

Summarize the Model Architecture

This cell prints a summary of the final model's architecture, including the layers, output shapes, and the number of parameters.

```
vgg16.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0

Finally, the **VGG16 model** is redefined using the Model() function, combining the pre-trained base and the newly added output layers. The vgg16.summary() function provides a detailed overview of the model's structure, including the number of layers, parameters, and connections.

3.3 :Model Compilation and Training:

After defining the VGG16-based model, it is compiled and trained using appropriate optimization and regularization techniques to improve performance and prevent overfitting.

The **Adam optimizer** is used with a learning rate of **0.0001**, which helps in faster and more stable convergence during training. The **loss function** used is **categorical cross-entropy**, suitable for multi-class classification problems. The **accuracy** metric is included to evaluate the model's performance after each epoch.

An **EarlyStopping** callback is implemented to monitor the **validation accuracy** during training. If the validation accuracy does not improve for **3 consecutive epochs**, the training process stops automatically, and the best model weights are restored. This helps prevent overfitting and reduces unnecessary computation.

Finally, the model is trained using the **fit()** function for **10 epochs**, with both training and validation datasets provided. The training process records performance metrics such as loss and accuracy, which can later be visualized to assess model learning behavior.

```
(module) keras
# ---------- Training ----------
from keras.callbacks import EarlyStopping
from keras.optimizers import Adam
opt=Adam(learning_rate=0.0001)

#defining early stopping callback
early_stopping=EarlyStopping(monitor='val_accuracy',patience=3,restore_best_weights=True)

#compile the model
vgg16.compile(loss='categorical_crossentropy',optimizer=opt,metrics=['accuracy'])

#Train the model with early stopping callback
history=vgg16.fit(train,validation_data=test,epochs=10,callbacks=[early_stopping])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `s
  self._warn_if_super_not_called()
Epoch 1/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 81s 4s/step - accuracy: 0.4322 - loss: 1.1499 - val_accuracy: 0.6667 - val_loss: 0.5908
Epoch 2/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 392ms/step - accuracy: 0.7307 - loss: 0.5781 - val_accuracy: 0.6667 - val_loss: 0.6046
Epoch 3/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 398ms/step - accuracy: 0.7764 - loss: 0.4189 - val_accuracy: 0.6795 - val_loss: 0.6248
Epoch 4/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 412ms/step - accuracy: 0.8222 - loss: 0.3466 - val_accuracy: 0.7051 - val_loss: 0.6442
Epoch 5/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 397ms/step - accuracy: 0.8697 - loss: 0.2918 - val_accuracy: 0.6923 - val_loss: 0.7462
Epoch 6/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 420ms/step - accuracy: 0.8494 - loss: 0.3566 - val_accuracy: 0.6154 - val_loss: 1.0429
Epoch 7/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 406ms/step - accuracy: 0.8568 - loss: 0.3267 - val_accuracy: 0.7308 - val_loss: 0.6591
Epoch 8/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 412ms/step - accuracy: 0.8711 - loss: 0.2848 - val_accuracy: 0.7564 - val_loss: 0.5474
```

4.Testing Model & Data Prediction:

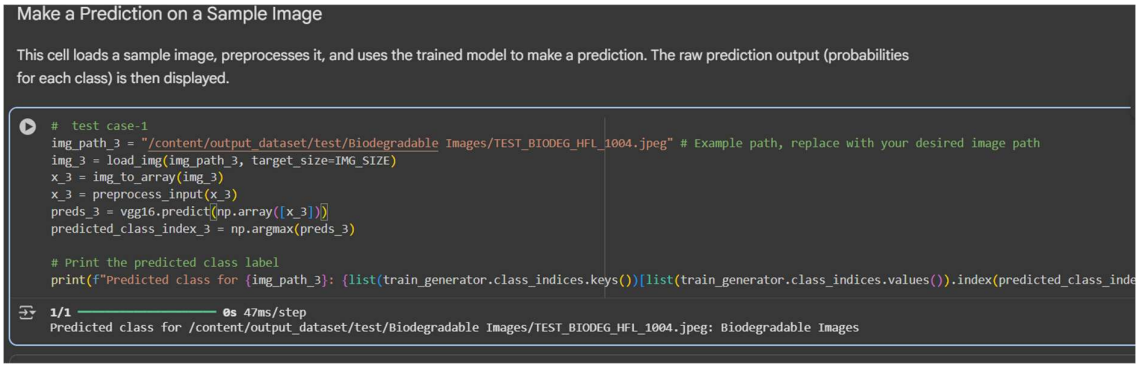
4.1:Testing the model

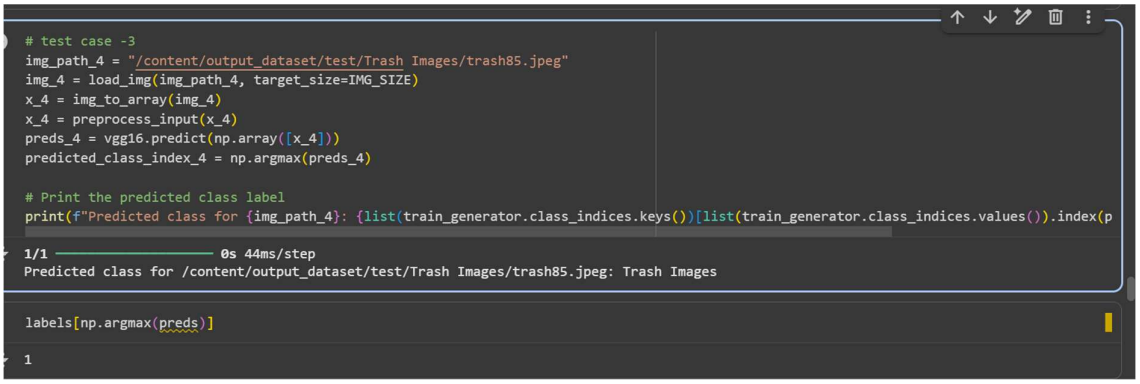
After training the VGG16-based model, its performance is first tested on individual images from the test dataset to verify real-time predictions. For each image:

1. The image is loaded from the file path and resized to 224 × 224 pixels.
2. It is converted to an array and preprocessed to match the model's input requirements.
3. The image is passed through the trained model to obtain prediction probabilities.
4. The class with the highest probability is selected as the predicted class, and the label is displayed.

This step ensures that the model correctly identifies images from all three classes (e.g., Biodegradable, Non-biodegradable, Recyclable) before saving.

Here we have tested with the Vgg16 Model With the help of the predict () function.

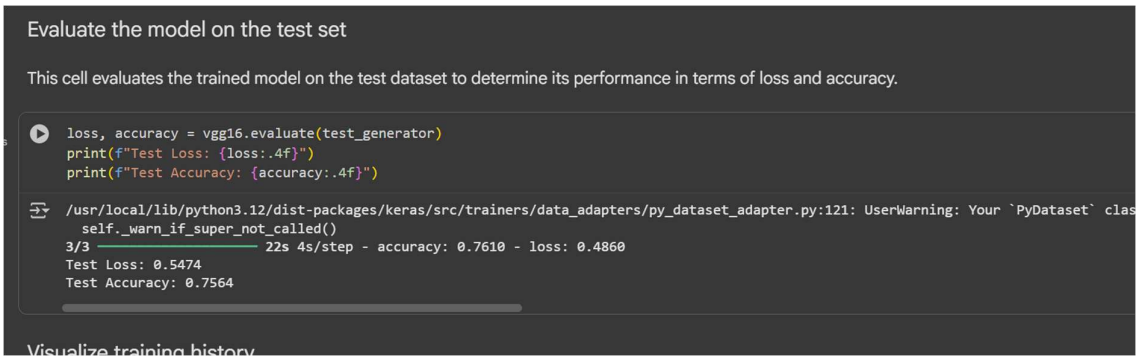




4.2:Model Evaluation on Test Dataset

After training, the model is evaluated on the **test dataset** to measure its performance on unseen images. The evaluate() function computes the **loss** and **accuracy** metrics, providing a quantitative measure of how well the model generalizes to new data.

In this project, the test dataset contains **three classes**, and evaluation ensures that the model can correctly classify images across all categories. The resulting **test loss** indicates how closely the predicted outputs match the true labels, while **test accuracy** reflects the proportion of correctly classified images. These metrics help assess the effectiveness of the trained VGG16-based model before deployment or saving for future use.



4.3:Training and Validation Performance Visualization

The training process of the VGG16-based model is monitored by plotting **accuracy** and **loss** for both training and validation datasets over the epochs.

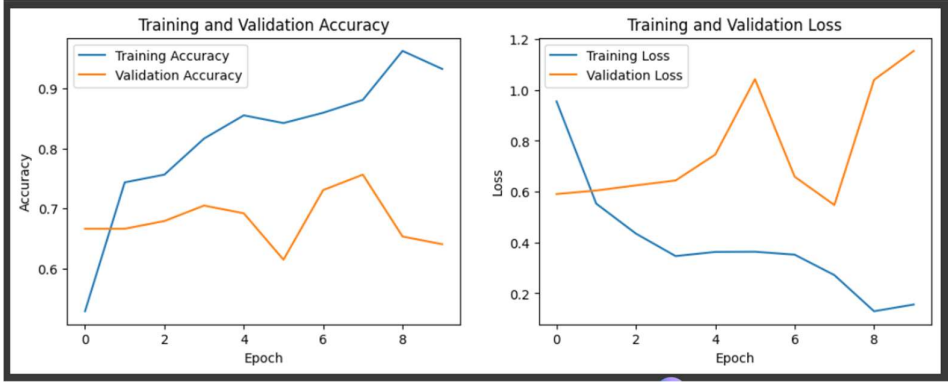
- The **accuracy plot** shows how the model's performance improves on the training dataset and how well it generalizes to the validation dataset.
- The **loss plot** indicates how the prediction error decreases over time, helping to identify underfitting or overfitting during training.

These plots provide a visual representation of the model's learning behavior, making it easier to assess convergence, evaluate performance, and make decisions about further tuning or training.

```
plt.legend()

# Plot training and validation loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



5.Saving the model:

After training and evaluating the VGG16-based model, it is saved for future use to avoid retraining. The model is stored in **HDF5 format (waste_classifier.h5)**, which preserves both the model architecture and learned weights.

In addition, the **class indices** (mapping between class names and their corresponding numeric labels) are saved to a **JSON file (class_indices.json)**. This ensures that during inference or deployment, predictions can be correctly mapped back to their original class labels.

By saving both the trained model and class indices, the system can **load the model later** and make predictions on new images efficiently, without repeating the training process.

Save Model and Class Indices

This cell saves the trained VGG16 model to an HDF5 file (`waste_classifier.h5`) and the class indices (mapping between class names and their numerical labels) to a JSON file (`class_indices.json`). These files can be used later to load the trained model and make predictions without retraining.

```
# Save the model
vgg16.save("waste_classifier.h5")

# Save class indices to a json file
import json
with open("class_indices.json", "w") as f:
    json.dump(train_generator.class_indices, f)
```

6.Application Building:

In this section, we will be building a web application that is integrated into the model we built. A UI is provided for the uses where they can upload image for predictions. The enter values are given to the saved model and prediction is showcased on the UI.

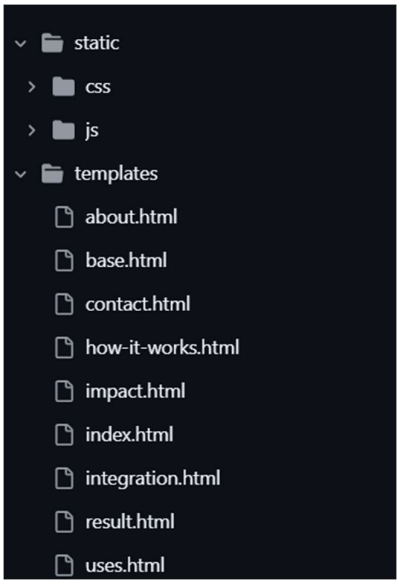
This section has the following tasks

- Building HTML Pages
- Building server-side script

6.1:Building HTML Pages:

For this project create HTML files namely

Building HTML Pages:



For this project create three HTML files namely

- index.html
- result.html
- base.html and all other supporting files

And save them in the templates folder.

6.2:Build Python code:

Import the libraries

```
import os
import json
import numpy as np
from flask import Flask, render_template, request, redirect, url_for, flash
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
from werkzeug.utils import secure_filename
# from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.applications.vgg16 import preprocess_input
```

Load the saved model. Importing the Flask module in the project is mandatory. An object of the Flask class is our WSGI application. The Flask constructor takes the name of the current module (__name__) as argument.

```
# Flask app
app = Flask(__name__)
app.config["UPLOAD_FOLDER"] = "static/uploads"
app.secret_key = "cleantech_secret_key"  # for flash messages

# Load model & class indices
model = load_model("waste_classifier.h5")
with open("class_indices.json") as f:
    class_indices = json.load(f)
class_labels = [class_indices[str(i)] for i in range(len(class_indices))]
```

Here we will be using the declared constructor to route to the HTML page which we have created earlier.

```
@app.route("/", methods=["GET", "POST"])
def home():
    if request.method == "POST":
        file = request.files.get("file")
        if file:
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
            file.save(filepath)
            label, confidence = predict_image(filepath)
            return render_template(
                "result.html",
                label=label,
                confidence=round(confidence*100,2),
                filename=filename
            )
    return render_template("index.html", title="Home")
```

In the above example, the '/' URL is bound with the index.html function. Hence, when the index page of the web server is opened in the browser, the html page will be rendered. Whenever you enter the values from the html page the values can be retrieved using POST Method.

Retrieves the value from UI:

```python
# Helper function: predict uploaded image
def predict_image(img_path):
    img = image.load_img(img_path, target_size=(224,224))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = preprocess_input(img_array)

    preds = model.predict(img_array)
    idx = np.argmax(preds[0])
    return class_labels[idx], float(np.max(preds[0]))
```

Here we are routing our app to the predict_image()function. This function retrieves all the values from the HTML page using a Post request. That is stored in an array. This array is passed to the model. Predict () function. This function returns the prediction. This prediction value will rendered to the text that we have mentioned in the result.html page earlier.

Main Function:

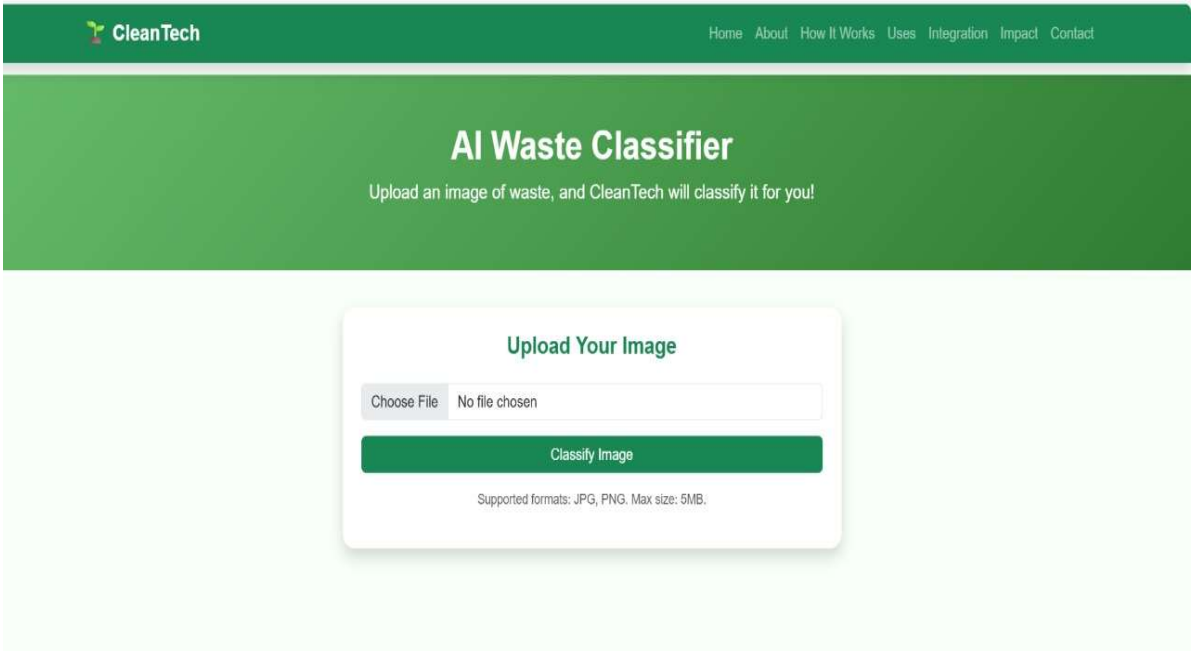
```python
if __name__ == "__main__":
    # This block is for local development.
    port = int(os.environ.get("PORT", 5000))
    # Use 0.0.0.0 to be accessible from the network, not just localhost.
    app.run(debug=False, host="0.0.0.0", port=port)
```

- **Run the web application**
- Run the application
- Open Anaconda prompt from the start menu
- Navigate to the folder where your Python script is.
- Now type the “python app.py” command
- Navigate to the local host where you can view your web page.
- Click on the inspect button from the top right corner, enter the inputs, click on the classify image button, and see the result/prediction on the web.

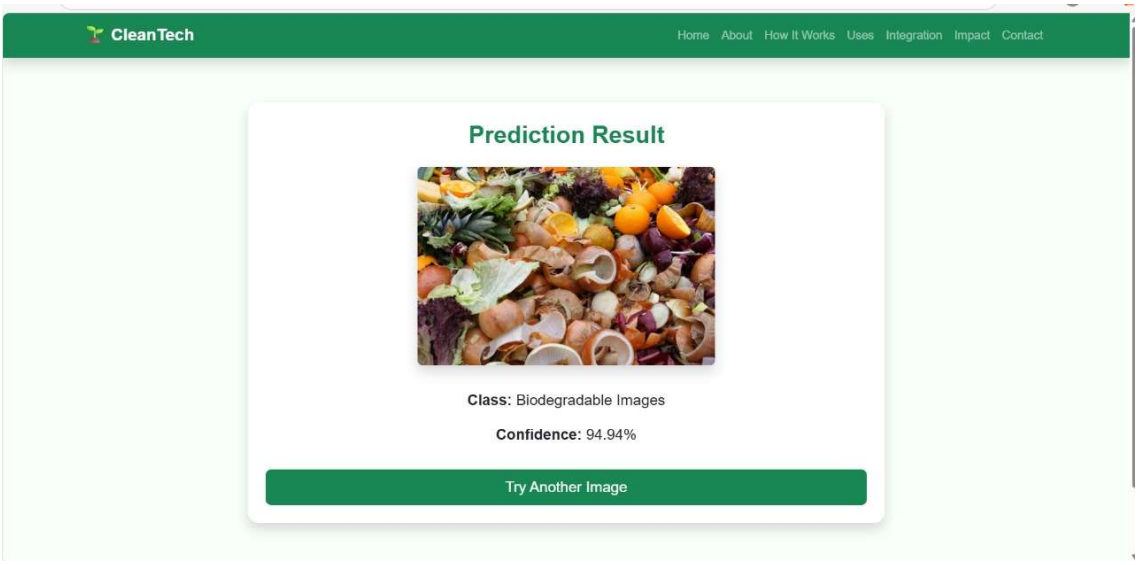
```
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFl
w with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty unt
l you train or evaluate the model.
 * Serving Flask app 'app'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inst
ad.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.146.22.133:5000
INFO:werkzeug:Press CTRL+C to quit
```

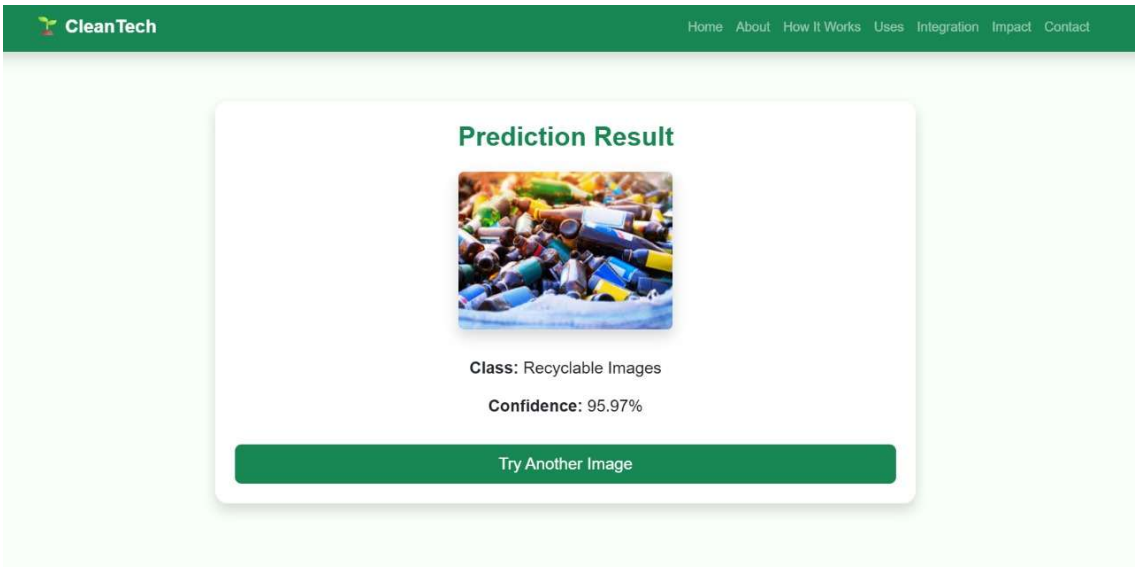
UI Image preview:

Main Application Page (Home Page): The main user interface of the CleanTech application is the home page, which serves as the entry point for waste classification. It features a clean design with a central upload module that prompts the user to choose an image file and submit it for analysis by clicking the "Classify Image" button.

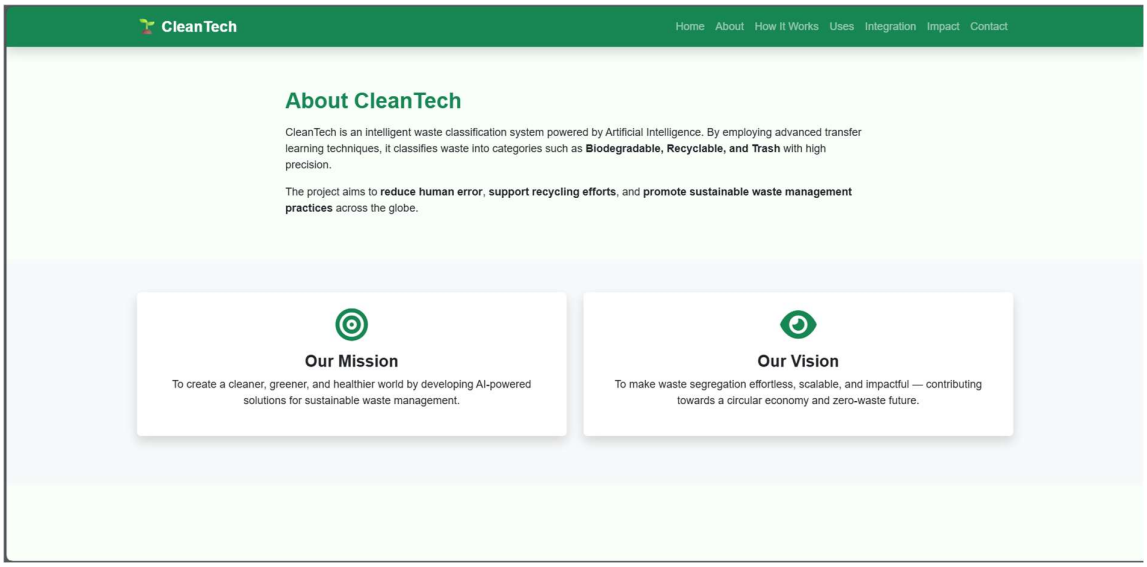


Prediction Result Page: After an image is submitted, the application displays the result page. This screen presents the uploaded image, the predicted waste category (e.g., Biodegradable, Recyclable), and the model's confidence score for that prediction. A "Try Another Image" button allows the user to return to the home page to perform another classification

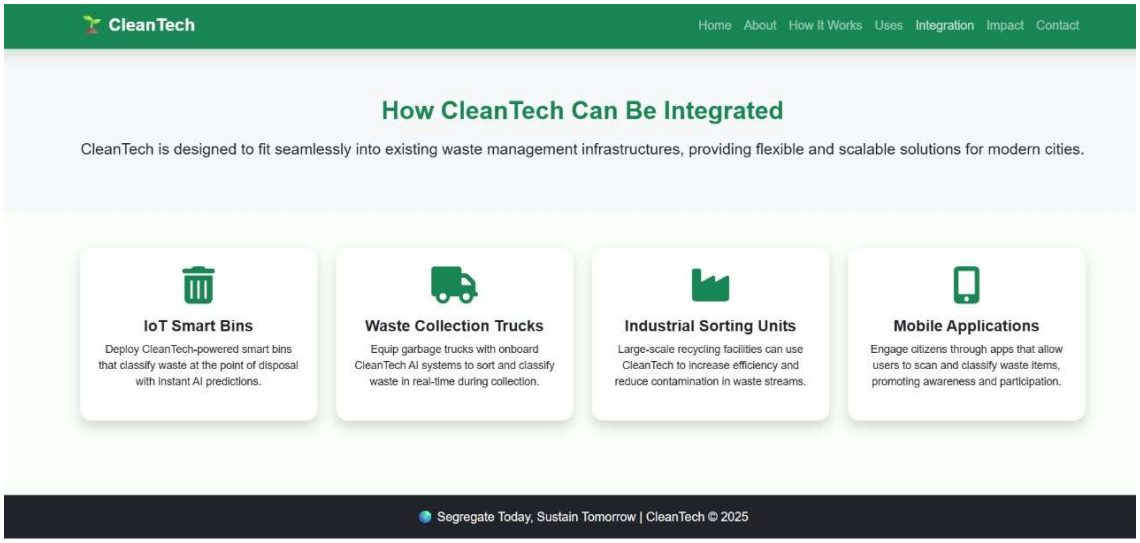




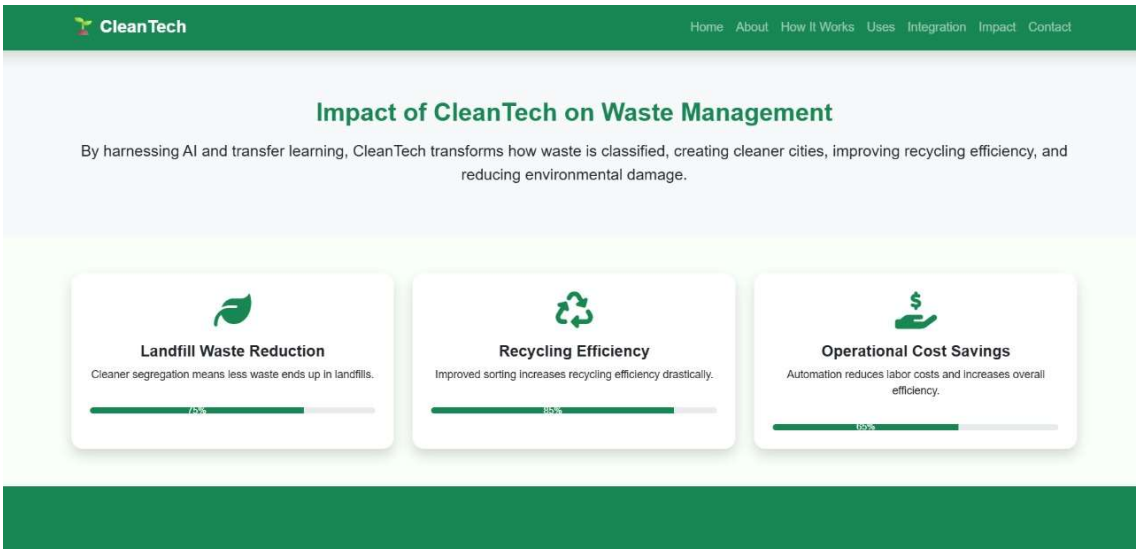
About Page: The "About CleanTech" page provides an overview of the project's mission and vision. It explains that the system is an intelligent waste classification tool powered by Artificial Intelligence and transfer learning techniques, aiming to promote sustainable waste management.



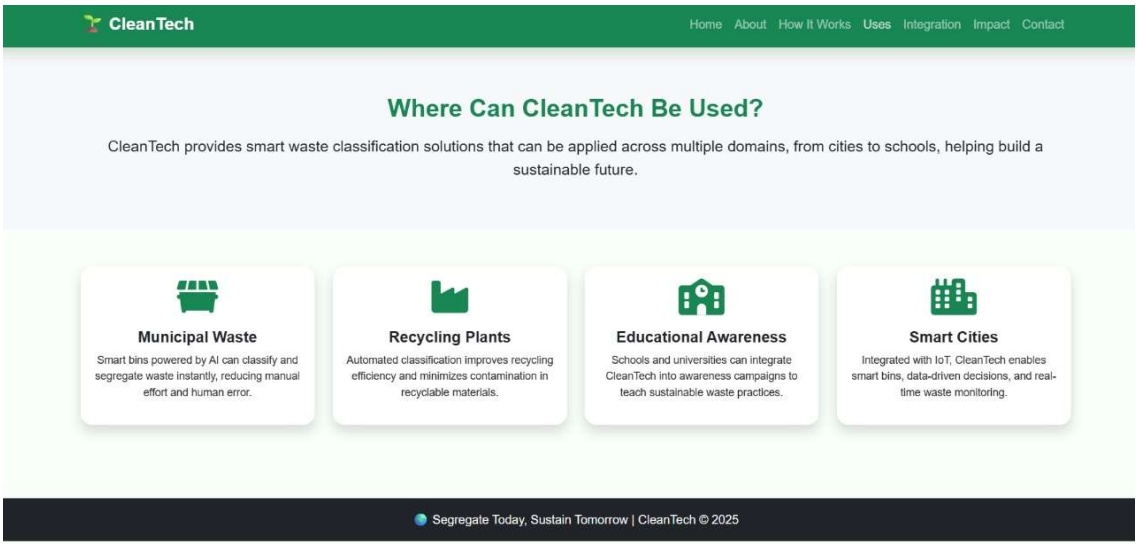
Integration Page: This page illustrates how the CleanTech system can be seamlessly integrated into existing infrastructures. It showcases several scalable solutions, including deployment in IoT Smart Bins, Waste Collection Trucks, Industrial Sorting Units, and Mobile Applications

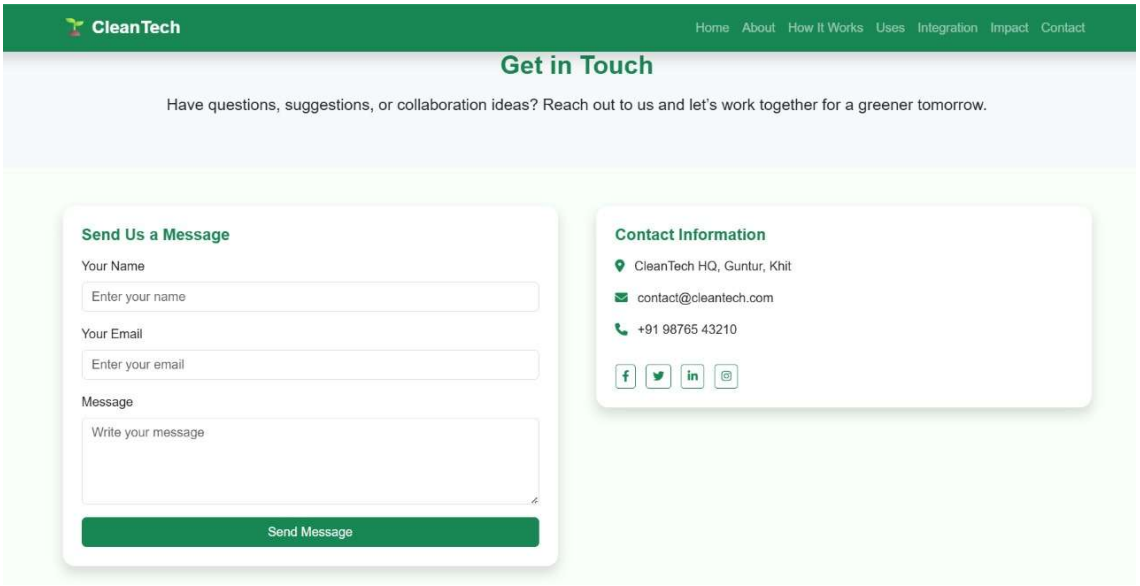


Impact Page: The "Impact" page highlights the benefits of the CleanTech system on waste management. It focuses on key improvements such as Landfill Waste Reduction, increased Recycling Efficiency, and Operational Cost Savings achieved through automation.



Uses Page: This page outlines the various domains where the CleanTech classification solution can be applied. Use cases include managing Municipal Waste, automating Recycling Plants, promoting Educational Awareness, and integrating with Smart Cities for real-time monitoring.





7. Conclusion

The CleanTech project successfully addressed the challenge of inefficient manual waste sorting by developing an intelligent classification system. The primary objective was to create a scalable and accurate tool for automated waste segregation by leveraging deep learning techniques.

By employing **transfer learning** with a pre-trained **VGG16 model**, the system was effectively trained on a custom dataset of three distinct waste categories: Recyclable, Biodegradable, and Trash. After training, the model achieved a final accuracy of **75.64%** on the unseen test dataset, demonstrating its capability to generalize and make reliable predictions.

The trained model was successfully integrated into a user-friendly web application built with Flask, demonstrating a practical and complete end-to-end solution. This project confirms that transfer learning is a highly effective approach for creating tools that can enhance recycling efficiency, reduce operational costs, and contribute to smarter, more sustainable waste management solutions. ♻️